

Aula 11: Herança Múltipla e Interfaces em Java

Programação Orientada a Objetos

Nesta aula, exploramos um dos pilares mais importantes da POO em Java: as interfaces. Entenderemos por que Java optou por não permitir herança múltipla de classes, como as interfaces resolvem esse problema de forma elegante e como utilizá-las para criar sistemas flexíveis, extensíveis e de fácil manutenção.

POO EM
JAVA

AULA 11

NÍVEL INICIANTE

Objetivos de Aprendizagem

Ao final desta aula, você será capaz de:

1

Compreender o Problema do Diamante

Identificar ambiguidades e conflitos gerados pela herança múltipla de classes.

2

Dominar Interfaces

Declarar, implementar e estender interfaces em Java com segurança e clareza.

3

Diferenciar Interface de Classe Abstrata

Escolher a construção correta para cada cenário de design.

4

Aplicar Boas Práticas

Usar o Princípio da Segregação de Interfaces (ISP) e recursos modernos do Java 8+.

O Problema do Diamante

HERANÇA MÚLTIPLA

O **Problema do Diamante** acontece quando uma classe tenta herdar de duas classes diferentes, mas essas duas classes vieram de uma mesma classe original.

Imagine assim: A **ClasseA** tem um método chamado `fazAlgo()`.

Depois, a **ClasseB** e a **ClasseC** herdam da **ClasseA** e cada uma cria sua própria versão desse método.

Agora, se a **ClasseD** tentar herdar ao mesmo tempo da **ClasseB** e da **ClasseC**, surge uma dúvida: *"Qual versão do método `fazAlgo()` a ClasseD deve usar? A da ClasseB ou a da ClasseC?"*

Esse é o problema. O compilador pode não saber qual método escolher, causando conflito e ambiguidade no programa. Por isso, algumas linguagens, como Java, não permitem herança múltipla entre classes.



ClasseA

Define `fazAlgo()` original



ClasseB e ClasseC

Ambas herdam de A e sobrescrevem o método



ClasseD

Herda de B e C — qual método usar? Ambiguidade fatal!

Por Que Java Não Permite Herança Múltipla de Classes?

A Decisão de Design

Os criadores do Java tomaram uma decisão consciente: **simplicidade e segurança acima de tudo**.

Linguagens como C++ permitem herança múltipla, mas isso traz complexidade enorme ao compilador e ao programador.

Java optou por eliminar essa fonte de bugs e ambiguidades.

- ❏ Em Java, uma classe pode herdar de **apenas uma classe** (usando `extends`), mas pode implementar **múltiplas interfaces**.

Problemas Evitados

Ambiguidade de métodos

1

Conflito na chamada de métodos com mesmo nome e assinatura

Ambiguidade de atributos

2

Campos com o mesmo nome vindos de classes diferentes

Complexidade de manutenção

3

Hierarquias difíceis de entender e refatorar

Interfaces: O Contrato de Comportamento

Uma **interface** em Java é um *contrato* que define um conjunto de comportamentos que uma classe deve implementar.

Ela especifica **o que** uma classe deve fazer, mas não **como** fazer. É como um contrato legal: a classe que assina (implementa) a interface se compromete a fornecer todas as funcionalidades ali descritas.

1 Contrato Formal

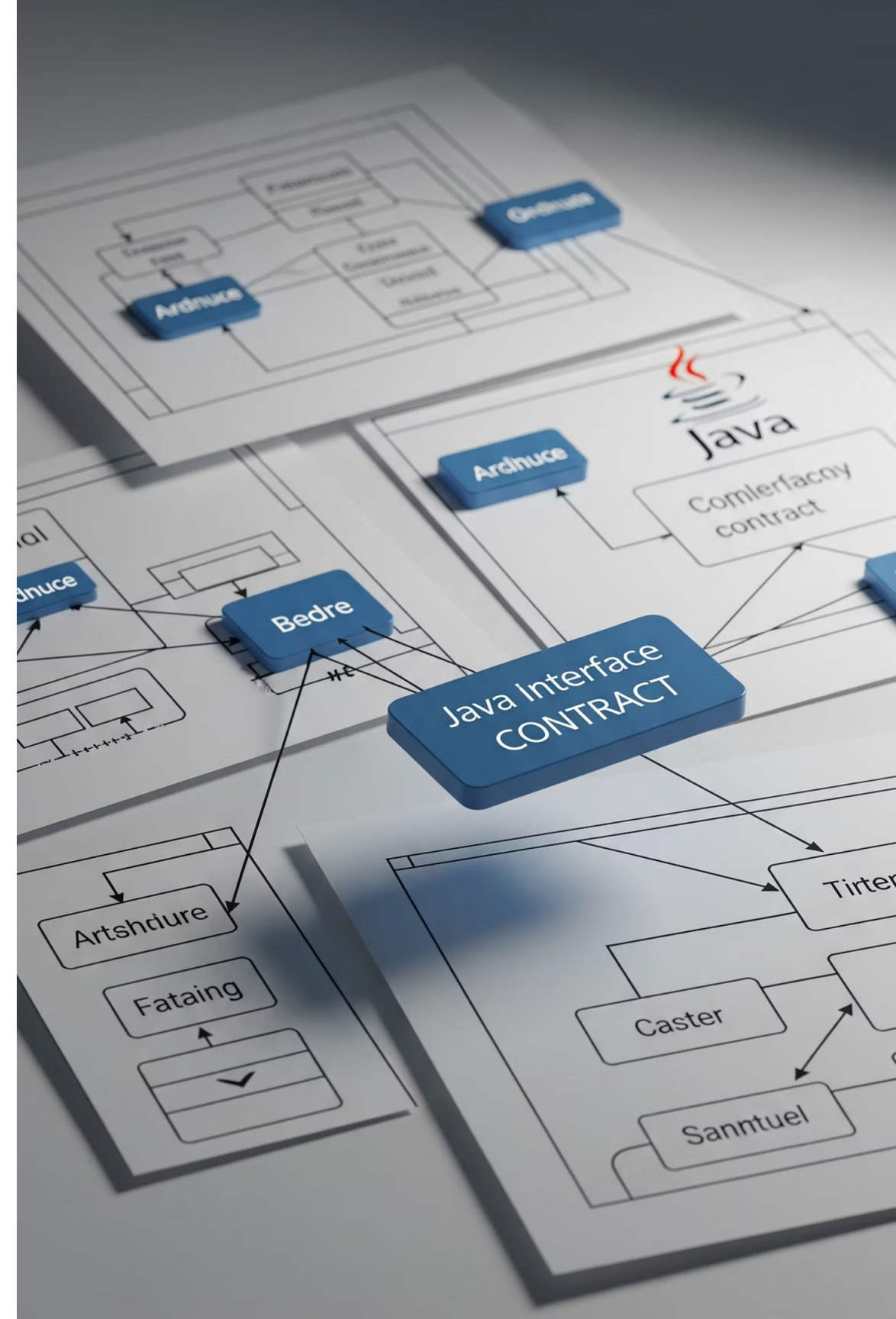
Define métodos que toda classe implementadora deve fornecer, garantindo um comportamento previsível e consistente.

2 Acoplamento Fraco

Permite que diferentes classes sejam tratadas de forma uniforme, sem depender da implementação concreta.

3 Herança Múltipla Segura

Uma classe pode implementar quantas interfaces quiser, obtendo os benefícios da múltipla herança sem os riscos.



Sintaxe Básica de Interface em Java

A palavra-chave `interface` define o contrato. Todos os métodos são, por padrão, `public abstract`. A classe que implementa usa `implements` e é obrigada a fornecer o corpo de todos os métodos.

Declarando a Interface

```
// Definição da interface
public interface Animal {
    // Método abstrato (sem corpo)
    String getNome();
    void emitirSom();

    // Constante (public static final)
    int MAX_VELOCIDADE = 300;
}
```

Todos os atributos declarados em interfaces são implicitamente `public static final` — ou seja, constantes. Métodos são implicitamente `public abstract`.

Implementando a Interface

```
// Classe que implementa Animal
public class Cachorro implements Animal {

    @Override
    public String getNome() {
        return "Cachorro";
    }

    @Override
    public void emitirSom() {
        System.out.println("Au au!");
    }
}
```

Interface vs. Classe Abstrata: Comparação Detalhada

Escolher entre interface e classe abstrata é uma decisão de design fundamental. Use a tabela abaixo como referência:

Característica	Interface	Classe Abstrata
Herança múltipla	✔ Sim (implements N)	✘ Não (extends 1)
Construtores	✘ Não possui	✔ Pode ter
Atributos de instância	✘ Apenas constantes	✔ Qualquer tipo
Métodos concretos	✔ default/static (Java 8+)	✔ Sempre permitido
Modificadores de acesso	Apenas public	Qualquer modificador
Quando usar	Definir capacidades (pode voar, pode nadar)	Compartilhar estado e comportamento base

Implementando Interfaces: `implements` e Múltiplas Interfaces

Uma das maiores vantagens das interfaces em Java é que uma única classe pode implementar **várias interfaces simultaneamente**, simulando a herança múltipla de forma segura. Basta listar as interfaces separadas por vírgula após a palavra-chave `implements`.

```
// Interfaces separadas
public interface Voador {
    void voar();
}

public interface Nadador {
    void nadar();
}

public interface Corredor {
    void correr();
}
```

```
// Classe implementando MÚLTIPLAS interfaces
public class Pato implements Voador, Nadador,
Corredor {

    @Override
    public void voar() {
        System.out.println("Pato voando...");
    }

    @Override
    public void nadar() {
        System.out.println("Pato nadando...");
    }

    @Override
    public void correr() {
        System.out.println("Pato correndo mal...");
    }
}
```



O pato é um exemplo clássico: ele *pode voar*, *pode nadar* e *pode correr* — capacidades distintas que, em um design orientado a interfaces, ficam separadas e reutilizáveis.

Herança entre Interfaces: `extends`

Assim como classes herdam de classes, **interfaces também podem herdar de outras interfaces** usando `extends`. Uma interface pode inclusive estender **múltiplas interfaces** ao mesmo tempo — algo impossível com classes. Isso cria hierarquias de contratos flexíveis e reutilizáveis.

```
// Interface base
public interface SerVivo {
    void respirar();
    void metabolizar();
}

// Interface que herda de SerVivo
public interface Animal extends SerVivo {
    void locomover();
    String getNome();
}

// Interface herdando de múltiplas
public interface PetDomestico extends Animal,
Domesticavel {
    void interagirComHumano();
}
```

Regras Importantes

1

Interface herda contrato completo

Todos os métodos da interface pai ficam disponíveis na filha

2

Classe implementa tudo

Quem implementa a interface filha deve implementar TODOS os métodos, incluindo os herdados

3

Extends múltiplo é permitido

Interface pode estender várias interfaces ao mesmo tempo

Métodos `default` e `static` em Interfaces (Java 8+)

No Java 8+, interfaces podem ter alguns métodos com corpo. Isso permite adicionar comportamentos prontos sem obrigar todas as classes a implementarem tudo do zero.

Método `default`

Um método `default` já vem pronto na interface. A classe que implementa a interface pode: **usar o método como ele está** ou **sobrescrever o método se quiser outro comportamento**

```
public interface Cumprimentador {  
    default void cumprimentar(String nome){  
        System.out.println("Olá, " + nome + "!");  
    }  
    void despedir(String nome);  
}  
class CumprimentadorFormal implements Cumprimentador  
{  
    @Override  
    public void despedir(String nome) {  
        System.out.println("Até logo, " + nome);  
    }  
    // Não precisa implementar cumprimentar()  
    // porque ele já existe como default  
}
```

Método `static`

Um método `static` pertence à própria interface. Ele não é herdado pela classe e deve ser chamado usando o nome da interface.

```
public interface Validador {  
    boolean validar(String valor);  
    // Método utilitário estático  
    static boolean isNullOrEmpty(String s) {  
        return s == null || s.trim().isEmpty();  
    }  
}  
  
// Uso do método estático  
// Chamado diretamente pela interface!  
boolean vazio = Validador.isNullOrEmpty(" ");  
// resultado: true
```

Interfaces Funcionais e Expressões Lambda

Uma **interface funcional** é qualquer interface que possua ***exatamente um método abstrato***.

Como existe somente um método para implementar, o Java permite usar uma **expressão lambda** para escrever essa implementação de forma mais curta.

Declaração

```
@FunctionalInterface
public interface Operacao {
    int executar(int a, int b);
}
```

new Operacao(){...}

Aqui parece que estamos instanciando uma interface, mas **não é exatamente isso**. Interface não pode ser instanciada diretamente.

O que o Java faz aqui é: Criar uma classe sem nome, que implementa **Operacao**. Esse método é conhecido como **classe anônima**.

Uso com Lambda

```
// Sem lambda (verboso)
Operacao soma = new Operacao() {
    public int executar(int a, int b) {
        return a + b;
    }
};
```

```
// Com lambda (conciso e elegante)
Operacao soma = (a, b) -> a + b;
Operacao mult = (a, b) -> a * b;
```

```
System.out.println(soma.executar(3, 4)); // 7
System.out.println(mult.executar(3, 4)); // 12
```

Sistema de Pagamentos

```
public interface Pagavel {
    boolean processar(double valor);
    String getDescricao();
}

public class CartaoCredito implements Pagavel {
    private String numero;

    @Override
    public boolean processar(double valor) {
        System.out.println("Cobrando R$"
            + valor + " no cartão");
        return true; // aprovado
    }
    @Override
    public String getDescricao() {
        return "Cartão: " + numero;
    }
}

// Método polimórfico - não importa o tipo!
public void efetuarPagamento(Pagavel meio,
                               double valor) {
    if (meio.processar(valor)) {
        System.out.println("Pago via: "
            + meio.getDescricao());
    }
}
```

Sistema de Animais

```
public interface Voador {
    void voar();
    default String getTipoVoo() {
        return "voo padrão";
    }
}

public interface Aquatico {
    void mergulhar();
}

// Pássaro só voa
public class Passaro implements Voador {
    public void voar() {
        System.out.println("Batendo asas!");
    }
}

// Pato: voa E nada!
public class Pato
    implements Voador, Aquatico {
    public void voar() {
        System.out.println("Voo curto do pato");
    }
    public void mergulhar() {
        System.out.println("Mergulhando!");
    }
}
```

Interfaces Importantes da API Java

A biblioteca padrão do Java é construída sobre interfaces poderosas. Conhecê-las é essencial para escrever código idiomático e integrado ao ecossistema Java.



Comparable<T>

Permite que objetos sejam **ordenados naturalmente**.

Implementar `compareTo(T o)` possibilita o uso em `Collections.sort()` e estruturas como `TreeSet`.



Runnable

Interface funcional para definir tarefas executadas por **threads**. O método `run()` contém o código a ser executado de forma concorrente.



Cloneable e Serializable

`Cloneable` habilita o método `clone()` para cópia de objetos. `Serializable` permite a **serialização** — transformar objetos em bytes para armazenamento ou transmissão em rede.

Princípio da Segregação de Interfaces (ISP)

SOLID

O **Interface Segregation Principle** — o "I" do SOLID — afirma: *"Nenhuma classe deve ser forçada a implementar métodos que não utiliza."* Interfaces grandes e genéricas criam dependências desnecessárias. A solução é criar interfaces menores e mais focadas.

❌ Violando o ISP

```
// Interface "gorda" – problema!
public interface Trabalhador {
    void trabalhar();
    void comer();    // robô não come!
    void dormir();   // robô não dorme!
}

// Robô forçado a implementar
// métodos sem sentido:
public class Robo implements Trabalhador {
    public void trabalhar() { ... }
    public void comer() { /* vazio! */ }
    public void dormir() { /* vazio! */ }
}
```

✅ Respeitando o ISP

```
// Interfaces segregadas – solução!
public interface Trabalhavel {
    void trabalhar();
}

public interface Alimentavel {
    void comer();
}

public interface Descansavel {
    void dormir();
}
```

Humano implementa o que faz sentido: *Trabalhavel, Alimentavel, Descansavel*.

Robô implementa só o necessário: *Trabalhavel*